mediaCompMovie

Movie functions for mediaComp

Document based on mediaCompMovie (movies branch), for mediaComp v0.4.15
October 30, 2026

About this document

This document describes the movie functions of mediaCompMovie, an add-on to the mediaComp library for building an animation frame by frame, playing it in a window, and saving it as a video file. It follows the form of mediaComp's own Media Functions document: each function's signature, a description, its parameters and return value, and an example. The functions are listed alphabetically. Appendix A lists how they differ from the movie functions of JES, for anyone working from the Guzdial and Ericson textbook.

The following visual conventions are used in this document.

- Function signatures: optional parameters and punctuation are enclosed in brackets [ ].
- Descriptions: when referencing a parameter, the parameter is shown in *italics*.
- Example code is displayed in Courier font.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

Table of Contents

About this document....1
Table of Contents....2
Movie Functions....3
addFrameToMovie(frame, movie)....3
getMovieFormat()....4
makeMovie([directory])....4
makeMovieFromInitialFile(filename)....5
playMovie(movie[, isBlocking])....5
setMovieFormat(fmt)....6
writeAVI(movie, path[, framesPerSec])....7
writeFrame(num, directory, picture)....7
writeFramesToDirectory(movie[, directory])....8
writeMovie(movie, path[, framesPerSec])....8
writeMP4(movie, path[, framesPerSec])....9
writeQuicktime(movie, path[, framesPerSec])....9
Appendix A: Differences from the JES movie functions....10
playMovie(movie[, isBlocking])....10
Frame files....10
writeFramesToDirectory(movie[, directory])....10
addFrameToMovie(frame, movie)....10
writeAVI and writeQuicktime....10
Functions not in JES....10
Functions not provided....11

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

Movie Functions

These functions make, play and save movies. A movie is a sequence of pictures, called frames, shown one after another. To use them, copy `mediaCompMovie.py` into the same folder as your program, and start your program with these two lines:

```
from mediaComp import *
from mediaCompMovie import *
```

playMovie shows a movie in a window at 16 frames per second, the speed JES uses. Playing has no speed setting: the speed of a movie is chosen when it is saved as a video file, with the optional *framesPerSec* input of writeMovie, writeMP4, writeAVI and writeQuicktime.

writeFrame and writeFramesToDirectory save frames as PNG files named `frame0001.png`, `frame0002.png`, and so on. They replace files with the same names but leave other files alone. If an earlier run wrote 50 frames and this run writes 30, frames 31 to 50 are still in the folder, and `makeMovie(directory)` will include them. Delete the folder, or use a new one, before writing a shorter movie.

A file or folder name without a drive or full path, such as `"frames"`, is looked for in your media folder if you have set one with setMediaFolder. Otherwise it is looked for in the folder your program runs from, which in Thonny is normally your program's folder. One exception: given no folder, writeFramesToDirectory always saves to that current folder.

Saving a video with writeMovie, writeMP4, writeAVI or writeQuicktime needs two more packages, imageio and imageio-ffmpeg. In Thonny, install them with Tools → Manage packages. Without them, these functions print `Movie export needs: pip install imageio imageio-ffmpeg` and stop with an error. All the frames of a movie must be the same size to save it as a video. The saved videos play in Windows' own video player.

Every picture you add with addFrameToMovie is kept in memory. A 640x480 frame takes about 1 MB, so 100 frames take about 100 MB. For very long movies, use smaller pictures, or save the frames with writeFrame and make the movie with `makeMovie(directory)`.

---

addFrameToMovie(frame, movie)

Adds *frame* to the end of *movie*. *frame* can be a picture or the name of an image file. A picture is copied as it looks now, so you can keep drawing on the same picture and add it again for the next frame. The two inputs may be given in either order.

frame: the picture, or the name of an image file, to add
movie: the movie to add the frame to

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

returns: nothing

Example:
```
def makeGrowingSquareMovie():
    movie = makeMovie()
    canvas = makeEmptyPicture(200, 200)
    for size in range(10, 200, 10):
        addRectFilled(canvas, 0, 0, size, size, blue)
        addFrameToMovie(canvas, movie)
    return movie
```

This draws a bigger blue square on the same picture each time round the loop and adds the picture as the next frame. Because each frame is a copy, the movie shows the square growing.

---

getMovieFormat()

Tells you the video format writeMovie uses when the file name has no extension: `"mp4"`, `"avi"` or `"mov"`. It is `"mp4"` unless it has been changed with setMovieFormat. Takes no input.

returns: the format, as a string

Example:
```
def saveInChosenFormat(movie):
    print("Saving as", getMovieFormat())
    writeMovie(movie, "myMovie")
```

This prints the format writeMovie will use, then saves the movie as myMovie with that format's extension, for example `myMovie.mp4`.

---

makeMovie([directory])

Makes a new movie. With no input, the movie is empty: add frames to it with addFrameToMovie. With *directory*, the movie's frames are the image files (.png, .jpg or .jpeg) in that folder, in frame-number order. The folder is read each time the movie is played or saved, so frames written to it later are included.

directory: the folder that holds, or will hold, the frames (optional)
returns: the new movie

Examples:
```
movie = makeMovie()
```

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

```
print(movie)
```

This makes an empty movie and prints `Movie, frames: 0`.

```
def makeBallMovie():
    for num in range(1, 31):
        canvas = makeEmptyPicture(300, 200)
        addOvalFilled(canvas, num * 8, 80, 40, 40, red)
        writeFrame(num, "ball", canvas)
    return makeMovie("ball")
```

This saves 30 frames of a moving red ball in the ball folder, then returns a movie made from them.

---

makeMovieFromInitialFile(filename)

Makes a movie from a numbered sequence of image files, such as `frame0001.png`, `frame0002.png`, `frame0003.png`. Every file in the same folder whose name matches *filename* apart from its numbers becomes a frame, in number order. Unlike `makeMovie(directory)`, the files are found once, when the movie is made.

filename: the name of one of the frame files, for example "frames/frame0001.png"
returns: the new movie

Example:

```
def openFrames():
    file = pickAFile()
    return makeMovieFromInitialFile(file)
```

This lets the user pick a frame file, then returns a movie of all the files in that folder that are numbered like it.

---

playMovie(movie[, isBlocking])

Plays *movie* in a window, over and over, until the window is closed, at 16 frames per second. Press the space bar to pause or carry on; while paused, the left and right arrow keys step one frame back or forward. Escape closes the window. The window's title shows which frame is on screen, for example `Movie: frame 3 of 29`. The window shows the movie as it was when playMovie was called.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

Your program carries on while the movie plays, unless *isBlocking* is True: then playMovie waits until the window is closed. *isBlocking* is False if not given. Pass True to watch several movies one after another, or when running your program outside Thonny, where the window closes as soon as the program ends.

movie: the movie to play
isBlocking: True to wait until the window is closed (optional)
returns: nothing

Examples:

```
def showSquareMovie():
    movie = makeMovie()
    for num in range(1, 30):
        canvas = makeEmptyPicture(300, 200)
        addRectFilled(canvas, num * 10, num * 5, 50, 50, red)
        addFrameToMovie(canvas, movie)
    playMovie(movie)
```

This makes a movie of a red square moving down and to the right, and plays it. The function returns while the movie is still playing.

```
def playEach(movies):
    for movie in movies:
        playMovie(movie, True)
```

This plays each movie in the list in turn: each one starts when the window of the one before is closed. Without True, all the windows would open at once.

---

setMovieFormat(fmt)

Sets the video format writeMovie uses when the file name has no extension: `"mp4"`, `"avi"` or `"mov"`. The format starts as `"mp4"`. Capital letters and a leading dot are allowed, so `".AVI"` is the same as `"avi"`.

fmt: the format, "mp4", "avi" or "mov"
returns: nothing

Example:

```
def saveAsAVI(movie):
    setMovieFormat("avi")
    writeMovie(movie, "myMovie")
```

This saves the movie as `myMovie.avi`.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

---

writeAVI(movie, path[, framesPerSec])

Saves *movie* as an AVI video file, adding .avi to *path* if it is not there. The video plays *framesPerSec* frames per second, 16 if not given. All frames must be the same size, and the folder must already exist. AVI files are much larger than MP4 files of the same movie. Needs the imageio and imageio-ffmpeg packages.

movie: the movie to save
path: the name of the video file, for example "anim.avi"
framesPerSec: the number of frames per second (optional)
returns: nothing

Example:

```
def saveSlowly(movie):
    writeAVI(movie, "slow.avi", 4)
```

This saves the movie as `slow.avi`, playing 4 frames per second.

---

writeFrame(num, directory, picture)

Saves *picture* as one numbered frame in *directory*: `frame0001.png` for *num* 1, `frame0002.png` for *num* 2, and so on. *num* must be a whole number, 0 or greater. The folder is made if it does not exist, and a frame file with the same name is replaced. Use `makeMovie(directory)` to make a movie from the saved frames.

num: the frame number
directory: the folder to save the frame in
picture: the picture to save
returns: nothing

Example:

```
def makeBallFrames():
    for num in range(1, 31):
        canvas = makeEmptyPicture(300, 200)
        addOvalFilled(canvas, num * 8, 80, 40, 40, blue)
        writeFrame(num, "frames", canvas)
    movie = makeMovie("frames")
    playMovie(movie)
```

This saves `frame0001.png` to `frame0030.png` in the frames folder, then plays them as a movie.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

---

writeFramesToDirectory(movie[, directory])

Saves every frame of *movie* as an image file in *directory*: `frame0001.png`, `frame0002.png`, and so on. The folder is made if it does not exist. Files with the same names are replaced; other files are left alone. If *directory* is not given, the frames are saved in the current folder, which in Thonny is normally your program's folder.

movie: the movie to save
directory: the folder to save the frames in (optional)
returns: nothing

Example:

```
def saveFrames(movie):
    writeFramesToDirectory(movie, "myFrames")
```

This saves each frame of the movie in the myFrames folder, where you can open them one at a time.

---

writeMovie(movie, path[, framesPerSec])

Saves *movie* as a video file. The format comes from the extension of *path*: .mp4, .avi or .mov. If *path* has no extension, the format set with setMovieFormat is used (MP4 unless it was changed) and its extension is added. The video plays *framesPerSec* frames per second, 16 if not given. All frames must be the same size, and the folder must already exist. Needs the imageio and imageio-ffmpeg packages.

movie: the movie to save
path: the name of the video file, for example "anim.mp4"
framesPerSec: the number of frames per second (optional)
returns: nothing

Example:

```
def saveTwice(movie):
    writeMovie(movie, "anim.mov")
    writeMovie(movie, "anim")
```

This saves the movie as the QuickTime file `anim.mov`, then as `anim.mp4`.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

writeMP4(movie, path[, framesPerSec])

Saves *movie* as an MP4 video file, adding .mp4 to *path* if it is not there. The video plays *framesPerSec* frames per second, 16 if not given. All frames must be the same size, and the folder must already exist. MP4 is the best choice for most movies: the files are small and play on almost any computer or phone. Needs the imageio and imageio-ffmpeg packages.

movie: the movie to save
path: the name of the video file, for example "anim.mp4"
framesPerSec: the number of frames per second (optional)
returns: nothing

Example:

```
def saveFast(movie):
    writeMP4(movie, "fast.mp4", 30)
```

This saves the movie as `fast.mp4`, playing 30 frames per second.

---

writeQuicktime(movie, path[, framesPerSec])

Saves *movie* as a QuickTime video file, adding .mov to *path* if it is not there. The video plays *framesPerSec* frames per second, 16 if not given. All frames must be the same size, and the folder must already exist. Needs the imageio and imageio-ffmpeg packages.

movie: the movie to save
path: the name of the video file, for example "anim.mov"
framesPerSec: the number of frames per second (optional)
returns: nothing

Example:

```
def saveForMac(movie):
    writeQuicktime(movie, "anim.mov")
```

This saves the movie as `anim.mov`, playing 16 frames per second.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

Appendix A: Differences from the JES movie functions

mediaCompMovie provides the movie functions of JES, the Jython Environment for Students, in Python 3. These are the differences that matter when running a program from the Guzdial and Ericson textbook or course notes.

playMovie(movie[, isBlocking])

playMovie takes no frame rate: it always plays 16 frames per second, and the speed of a movie is chosen when it is saved, for example with `writeMP4(movie, "anim.mp4", 30)`. It plays the movie over and over until the window is closed. It does not wait for the window to be closed unless True is given as *isBlocking*. Some versions of the movie library accept `playMovie(movie, delay)`, the time to show each frame; that is not supported. A second input is read as *isBlocking*, so `playMovie(movie, 0.1)` waits for the window but still plays 16 frames per second.

Frame files

writeFrame and writeFramesToDirectory save PNG files with four-digit numbers, `frame0001.png`, `frame0002.png`, and so on. JES saved frames as JPEG files.

writeFramesToDirectory(movie[, directory])

With no *directory*, JES saved the frames in your home folder. Here they are saved in the current folder, which in Thonny is normally your program's folder.

addFrameToMovie(frame, movie)

In JES, *frame* had to be the name of an image file. Here it can also be a picture, which is copied as it looks when it is added.

writeAVI and writeQuicktime

In JES, these needed every frame of the movie to be an image file in one folder. Here a movie made with addFrameToMovie can be saved directly. Saving needs the imageio and imageio-ffmpeg packages.

Functions not in JES

`makeMovie(directory)` and `writeFrame(num, directory, picture)` come from Mark Guzdial's course notebooks. writeMP4, writeMovie, setMovieFormat and getMovieFormat are new.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.

Functions not provided

`explore(movie)` and openFrameSequencerTool are not provided. To look at a movie frame by frame, play it with playMovie, press the space bar to pause, and step with the arrow keys.

This work © 2026 by Jeremy Porter is licensed under CC BY-NC-SA 4.0.